

Pesquisa aprofundada: testes destrutivos do runtime em um Docker-like system from scratch com Python, Cython e C++

Sleep, shell interativa, fork bomb, OOM, zombie reaping e signal forwarding

Relatório técnico para o contexto de um runtime “Docker-like” em Python, Cython e C++, com foco em *sleep*, shell interativa, fork bomb, OOM, zombie reaping e signal forwarding.

Data de elaboração: 12 mar. 2026

Resumo

Esta pesquisa parte de uma premissa simples: em um runtime de contêineres, os comportamentos mais importantes não aparecem quando tudo está funcionando, mas quando o workload tenta quebrar o contrato do runtime. Um sistema “Docker-like” feito do zero não se prova apenas ao conseguir executar `/bin/echo`; ele se prova quando mantém autoridade operacional sobre um processo que dorme indefinidamente, sobre um shell interativo com TTY, sobre uma explosão de processos, sobre pressão de memória que ameaça o host, sobre órfãos que virariam zumbis e sobre sinais que precisam atravessar múltiplas camadas sem ambiguidade. Nessa perspectiva, os cenários pedidos — *sleep*, shell interativa, fork bomb, OOM, zombie reaping e signal forwarding — não são casos periféricos de QA. Eles definem o núcleo da semântica operacional do runtime.

No Docker original, esses problemas aparecem em pontos diferentes da arquitetura e da UX. A interface de execução expõe `-i`, `-t`, `--init`, `--pids-limit`, sinais de parada e limites de memória; por baixo, o ecossistema converge para uma separação entre daemon de alto nível, shim por contêiner e runtime OCI, com forte dependência de primitivas do kernel como namespaces, cgroups, pseudoterminais, *pidfds* e mecanismos de espera e reap de processos [1][2][3][4][7][8][9][12][13][15][16][17][18][19]. Em um projeto cujo control plane e a modelagem de domínio ficam em Python, cujo data plane e as abstrações de hardware/kernel ficam em C++, e cujo Cython serve de cola, essa separação precisa ser feita de forma ainda mais explícita: o lado Python deve ser dono do modelo, do metadado, da API e da reconciliação; o lado C++ deve ser dono das operações privilegiadas, do caminho crítico de latência, dos FDs, do ciclo de vida do processo e das integrações com o kernel; o Cython deve ser um limite estreito, tipado e estável entre os dois [21][22][23].

A recomendação central deste documento é arquitetar o recurso como uma capacidade de produto, e não como uma suíte de testes jogada por fora. Em termos de System Design, isso significa um runtime agent em C++ isolado do processo principal da API, um *init shim* mínimo rodando como PID 1 dentro do namespace do contêiner, uma camada de orquestração e metadados em Python, e uma trilha de eventos rica o suficiente para

explicar não apenas que um contêiner morreu, mas *como* ele morreu. Em termos de Low Level Design, isso implica adotar cgroup v2 como baseline, monitorar *memory.events* e *pids.events*, usar *pidfd_open/pidfd_send_signal* quando disponíveis, consolidar sinais em laços baseados em *signalfd* e *epoll*, controlar PTYs via *openpty* e tratar limpeza agressiva com escalonamento de TERM para KILL e, em último caso, *cgroup.kill* [12][15][16][17][19]. Em termos de DDD, o sistema deve tratar “Sandbox”, “Init”, “ExecSession”, “ResourceEnvelope”, “TerminationIntent”, “ScenarioRun” e “Artifact” como partes explícitas da linguagem ubíqua. Em termos de padrões GoF, os pontos fortes são *Facade*, *Builder*, *State*, *Strategy*, *Observer*, *Command*, *Adapter* e *Template Method*.

O aspecto mais importante desta análise é talvez o menos intuitivo: para esta feature, os testes destrutivos e a implementação do runtime são praticamente a mesma conversa. O suporte a shell interativa exige PTY e semântica de terminal; o suporte a fork bomb exige limites de *pids* e limpeza confiável; o suporte a OOM exige modelagem correta de *memory.high*, *memory.max* e do que conta como falha de memória; o suporte a zombie reaping exige um PID 1 funcional; e o suporte a signal forwarding exige separar o caminho de sinais do operador do caminho de sinais gerados pelo terminal. Não existe caminho curto. O bom desenho do produto já precisa nascer sob essas pressões.

Sumário

1. Introdução	4
2. Docker original, OCI e primitivas do kernel	5
2.1. O que o Docker já ensina sobre esse problema	5
2.2. OCI como contrato de compatibilidade conceitual	6
2.3. As primitivas do kernel que realmente importam	6
3. O que significa tratar testes destrutivos como feature de runtime	7
4. Proposta de arquitetura para o projeto	7
4.1. Decisão macro: Python como dono do modelo, C++ como dono do kernel path	7
4.2. Componentes recomendados	8
4.3. Modelo de domínio e DDD	9
5. Análise aprofundada de cada conceito	10
5.1. Sleep	10
5.2. Shell interativa	11
5.3. Fork bomb	12
5.4. OOM	13
5.5. Zombie reaping	14
5.6. Signal forwarding	15
6. Plano de implementação em baixo nível	16
6.1. Contratos entre Python, Cython e C++	16
6.2. Runtime agent, init shim e laço de supervisão	17
6.3. PTY, sessões interativas e semântica de terminal	18
6.4. Observabilidade, artefatos e reconciliação	19
7. System Design, DDD e padrões de projeto aplicados	20
8. Requisitos funcionais e não funcionais	21
9. Roteiro incremental de implementação	23
10. Conclusão	24
Referências	24

1. Introdução

Construir um sistema “Docker-like” do zero costuma começar pela capacidade de criar namespaces, montar um root filesystem, aplicar alguns cgroups e dar `execve` em um processo. Isso é necessário, mas está longe de ser suficiente. O ponto em que um runtime deixa de ser apenas um lançador de processos e passa a ser um sistema operacional parcial está justamente naquilo que acontece depois do `exec`: como a plataforma mantém controle quando o workload não termina, quando precisa de terminal, quando multiplica processos de maneira patológica, quando esgota memória, quando abandona filhos e quando recebe sinais concorrentes? Esta pesquisa trata dessas perguntas como um recorte de arquitetura de produto, e não como notas de implementação isoladas.

No contexto do projeto proposto, a divisão tecnológica faz sentido e é coerente com a natureza do problema. Python é adequado para o control plane porque oferece alta velocidade de iteração, modelagem expressiva, ecossistema maduro para APIs e persistência, e capacidade de codificar políticas de orquestração e reconciliação com clareza. C++ é adequado para o data plane porque o caminho quente do runtime lida com FDs, syscalls, semântica de processo, PTYs, cgroups e sinais, e se beneficia de baixa sobrecarga, RAII, controle preciso de memória e integração natural com o kernel. O Cython, por sua vez, é um bom intermediário porque permite expor uma fronteira tipada e eficiente ao mundo Python sem obrigar o projeto a descer diretamente ao C API do interpretador em toda parte [21][22][23].

Mas a separação de linguagens não resolve automaticamente a separação de responsabilidades. Pelo contrário: ela a torna mais importante. Se o controle de estado está em Python, mas a verdade operacional sobre PIDs, sinais, TTYs e cgroups está em C++, então a semântica de vida e morte do contêiner precisa ser modelada com muita disciplina. Um erro frequente em projetos híbridos desse tipo é deixar o Python comandar demais o caminho quente, ou deixar o C++ vazar detalhes demais para cima. O efeito costuma ser dupla fonte de verdade, estados não reconciliáveis, corridas entre eventos de kernel e persistência, e dificuldade para explicar por que um contêiner ficou “travado” ou “sumiu”. O recorte desta feature é útil justamente porque força a arquitetura a responder às perguntas mais difíceis desde cedo.

Os seis conceitos pedidos têm papéis distintos. *Sleep* valida o happy path mínimo do ciclo de vida e a autoridade de parada. A shell interativa valida PTY, fluxo bidirecional, `resize`, `detach` e semântica terminal. A fork bomb valida contenção de PIDs, responsividade do supervisor sob pressão e limpeza de árvores de processo. O cenário de OOM valida limites, sinalização de causa, previsibilidade de pressão e preservação do host. Zombie reaping valida a qualidade do PID 1 e a limpeza pós-execução. Signal forwarding valida o contrato operacional entre cliente, control plane, runtime agent, init interno e workload. Em conjunto, eles cobrem quase toda a “mecânica invisível” que diferencia um runtime confiável de um protótipo.

2. Docker original, OCI e primitivas do kernel

2.1. O que o Docker já ensina sobre esse problema

O Docker moderno já materializa várias das decisões que este projeto terá de tomar. A documentação oficial registra que o Docker Engine usa o *containerd* para gerenciar o ciclo de vida dos contêineres e que, por padrão, o *containerd* usa o *runc* como runtime [1]. Essa camada intermediária não é detalhe histórico; ela existe porque o problema real não é apenas criar um contêiner, mas supervisioná-lo ao longo do tempo. O documento de ciclo de vida do *containerd* é ainda mais didático: cada contêiner possui um shim próprio que atua como pai direto dos processos do contêiner, mantém STDIO e/ou o PTY master abertos, registra status de saída e faz o reap dos processos quando eles terminam, inclusive permitindo que o daemon principal caia sem “matar” os contêineres [2]. Isso é praticamente a justificativa arquitetural para não embutir toda a lógica de supervisão privilegiada dentro do processo principal da API.

O README do Runtime v2 do *containerd* acrescenta uma sutileza decisiva para o tema desta pesquisa: o shim pode agir como *subreaper*, mas quando o contêiner roda em um novo PID namespace, o próprio processo init dentro do namespace é quem deve limpar órfãos reparentados, especialmente em cenários envolvendo *setns* e operações de *exec* [3]. Em outras palavras, um supervisor do lado de fora não substitui um PID 1 funcional do lado de dentro. Essa distinção se torna crítica em cenários de fork bomb, zombie reaping e signal forwarding. Ela também mostra por que uma implementação que “funciona” para *sleep* pode falhar de forma desastrosa nos demais cenários.

A superfície de usuário do Docker ecoa esses problemas. O comando *docker run* oferece *-i* para manter STDIN aberto, *-t* para anexar um pseudo-TTY, *--init* para rodar um processo init como PID 1, *--pids-limit* para limitar a quantidade de processos e um conjunto de flags de memória e de proxy de sinais, incluindo *--sig-proxy* [7]. A documentação do *docker attach* deixa explícito que, com *--sig-proxy=true*, um CTRL-C pode virar SIGINT para o processo do contêiner, e lembra que um processo rodando como PID 1 dentro de um contêiner é especial para o Linux, ignorando sinais cuja ação padrão seria terminá-lo caso não tenha handlers apropriados [8]. Já a documentação do *docker stop* define uma semântica operacional clara: o runtime envia primeiro o sinal configurado de parada — derivado do *StopSignal* da imagem ou de configuração explícita — e, após um timeout, escala para SIGKILL [9]. Essas escolhas não são cosméticas. Elas são a interface pública de decisões internas complexas.

Há ainda uma lição adicional na referência de Dockerfile. Quando o comando de entrada é expresso em forma shell, o processo real do workload pode deixar de ser PID 1 e a parada pode degradar para um caminho forçado em que o runtime precisa enviar SIGKILL depois do timeout. A documentação mostra exatamente esse comportamento ao comparar o uso de *exec* com um *ENTRYPOINT* shell-form [10]. Para o projeto proposto, isso implica que as imagens e os bundles usados na suíte destrutiva não devem depender de wrappers ambíguos. O teste deve isolar a semântica do runtime, e não ficar contaminado por uma camada de shell que desvia sinais e altera a árvore de processos.

2.2. OCI como contrato de compatibilidade conceitual

A especificação de runtime da OCI fornece o vocabulário mínimo para o desenho. Ela define *bundle*, *config.json*, processo, ambiente de execução e ciclo de vida do contêiner [4][5]. A estrutura de processo inclui, entre outros campos, *terminal*, *args*, *env*, *cwd*, capacidades e limites POSIX, o que é suficiente para encaixar os cenários de shell interativa, execução em foreground e testes com limites de recursos [5]. A especificação Linux também descreve a troca de estado do processo do contêiner por socket UNIX com descritores enviados via *SCM_RIGHTS*, uma pista importante para a modelagem de interfaces internas com FDs e para a ideia de separar control plane e data plane por uma fronteira bem definida [6].

Adotar OCI não significa copiar o Docker. Significa reduzir o acoplamento semântico entre o projeto e uma implementação específica. Para esta feature, o ganho prático é grande. Um *ScenarioRun* pode gerar um bundle OCI mínimo e delegar sua execução ao data plane em C++, preservando no control plane a semântica de “o que desejo executar”, sem obrigá-lo a conhecer a mecânica completa de *clone*, montagem, PTY e escrita em cgroups. Isso permite que o núcleo da pesquisa se concentre nas propriedades destrutivas do runtime e não na serialização ad hoc de parâmetros.

2.3. As primitivas do kernel que realmente importam

O primeiro bloco de primitivas decisivo é o de isolamento e hierarquia de processos. *PID namespaces* isolam o espaço de PIDs e fazem com que o primeiro processo no novo namespace se torne o PID 1 daquele ambiente; esse processo passa a adotar órfãos e, se ele morrer, o kernel termina todos os processos do namespace com SIGKILL [13]. A mesma documentação explica que apenas sinais para os quais o init do namespace instalou handler podem ser entregues a ele por outros membros do namespace, com a exceção de SIGKILL e SIGSTOP vindos de namespaces ancestrais [13]. O segundo bloco é o de reaping em árvore: *PR_SET_CHILD_SUBREAPER* permite transformar um processo intermediário em reaper de descendentes órfãos, mecanismo útil para um shim do lado de fora [14].

O terceiro bloco é o da supervisão moderna por descritores. *pidfd_open()* cria um file descriptor estável para um processo, e o manual ressalta que esse tipo de FD pode ser usado tanto para enviar sinais quanto para esperar encerramento e monitorar o processo com *poll*, *select* ou *epoll* [15]. *pidfd_send_signal()*, por sua vez, fornece um caminho sem corridas de reutilização de PID para sinalizar o alvo correto [16]. Em um runtime novo, isso é preferível a uma arquitetura baseada apenas em inteiros de PID espalhados entre threads e camadas. O quarto bloco é o de sinais sincronizados por FD. *signalfd()* permite consolidar sinais em um fluxo lido por file descriptor, o que reduz a dependência de handlers assíncronos e encaixa naturalmente em laços dirigidos por *epoll* [17].

O quinto bloco é o do terminal. O manual de PTY explica que o lado slave se comporta como um terminal clássico e que um processo que espera um terminal pode ser dirigido por um programa que detém o lado master [18]. Já *openpty()* encapsula a obtenção do par master/slave e a configuração inicial de parâmetros e tamanho de janela [19]. Isso é exatamente o que um runtime precisa para suportar shell interativa de forma séria. O sexto bloco é o de cgroup v2. A documentação oficial define *memory.high* como limite de throttling sem OOM, *memory.max* como limite duro que pode invocar o OOM killer, *memory.events* como fonte de telemetria para *high*, *max*, *oom* e *oom_kill*,

memory.oom.group como mecanismo para matar o grupo inteiro como unidade indivisível, *pids.max* como limite duro do número de processos, *pids.events* como contador de estouro e *cgroup.kill* como alavanca de limpeza forçada da subárvore [12]. Sem esse conjunto, a suíte destrutiva seria cega e imprecisa.

3. O que significa tratar testes destrutivos como feature de runtime

Há uma diferença importante entre testar a aplicação e testar o runtime. Quando uma aplicação falha sob OOM, fork storm ou sinais mal encaminhados, a pergunta do operador não é “minha lógica de negócio estava correta?”, mas “a plataforma preservou isolamento, coletou evidências, devolveu uma causa inteligível e se manteve operacional?”. Testes destrutivos do runtime não existem para provar que o workload é robusto; eles existem para provar que a infraestrutura continua governável quando o workload não é robusto. Essa inversão de foco muda a modelagem inteira do problema.

No contexto de um projeto que ainda está sendo construído, a tentação natural é adiar essa conversa para “depois do MVP”. Seria um erro. Os cenários aqui analisados não são um apêndice do sistema; eles moldam as decisões fundamentais de quem é o dono do PID 1, quem mantém o PTY master, onde os sinais são recebidos e retransmitidos, como os limites de recursos são escritos e observados, como a morte do workload é distinguida da morte do supervisor e como os artefatos de falha são persistidos. Se essas decisões forem tomadas sem o filtro dos cenários destrutivos, o acoplamento posterior tende a ser caro e doloroso.

A consequência prática é que a suíte destrutiva deve ser pensada como um subsistema de verificação incorporado ao produto. O control plane precisa entender o conceito de cenário, run, expectativa e artefato. O data plane precisa expor eventos suficientes para que tais cenários façam sentido. E a modelagem de domínio precisa representar estados e intenções de parada de forma mais rica do que “start” e “stop”. Essa postura evita que o sistema acabe com uma bateria externa de scripts incapaz de explicar estados internos, e cria uma ponte direta entre confiabilidade operacional, metadados e desenho de API.

4. Proposta de arquitetura para o projeto

4.1. Decisão macro: Python como dono do modelo, C++ como dono do kernel path

A arquitetura recomendada é composta por três zonas. A primeira é o control plane em Python, responsável por API, autenticação, persistência, reconciliação, catálogo de bundles, políticas, agendamento e interpretação semântica dos eventos. A segunda é o data plane em C++, responsável por operações privilegiadas, criação de namespaces, escrita em cgroups, gerenciamento de PTY, supervisão de processos, propagação de sinais, observação de FDs e limpeza. A terceira é a fronteira Cython, responsável por serialização tipada, mapeamento de erro, liberação de GIL em chamadas bloqueantes e exposição de uma API pequena e estável ao mundo Python [21][22][23].

A recomendação não é carregar um grande bloco de C++ dentro do mesmo processo da API HTTP. Embora isso seja tecnicamente possível via Cython, seria um desenho frágil para uma feature que lida justamente com estresse e comportamento patológico. Um bug de memória, um deadlock no caminho de sinais ou um laço bloqueante em C++ não deve

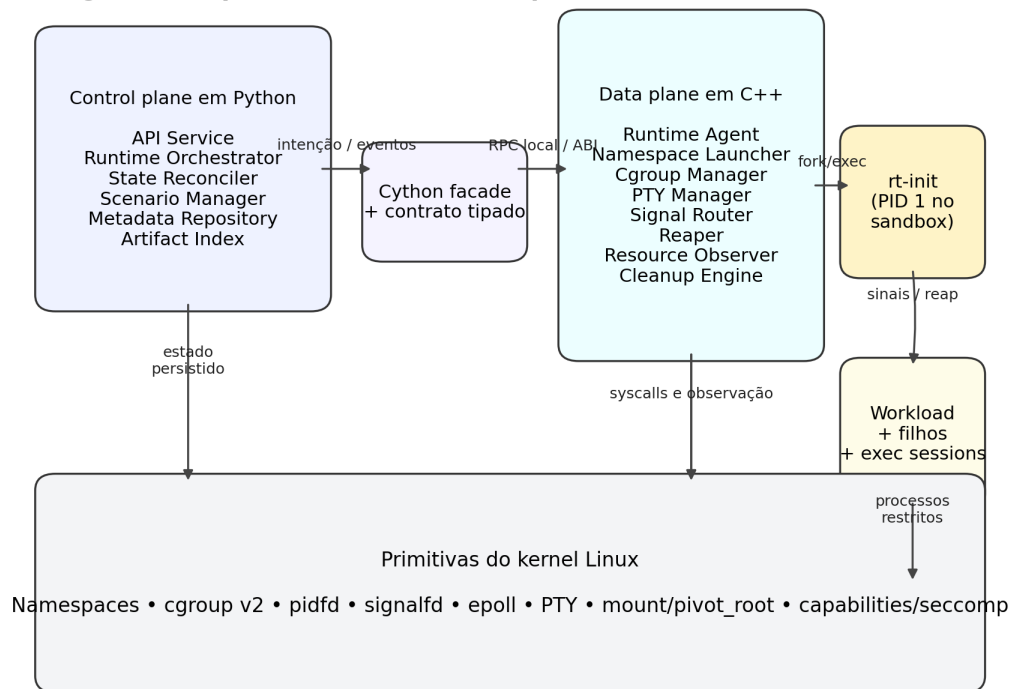
derrubar o serviço de metadados e orquestração. O desenho mais saudável é manter um *runtime agent* em C++ em processo separado, eventualmente supervisionado por um pequeno worker em Python que usa o módulo Cython para falar com ele por socket UNIX ou por uma ABI local estreita. O requisito “Cython como cola” continua atendido, mas a falha nativa fica confinada fora do coração do control plane.

Nesse modelo, o Python trata estado desejado e estado observado como conceitos de domínio. Ele não manipula diretamente PIDs, descritores ou semântica de PTY; ele solicita intenções: iniciar um sandbox com determinado *ResourceEnvelope*, anexar uma sessão interativa, enviar um sinal com determinada política de escalonamento, executar um cenário destrutivo e persistir artefatos. O C++ implementa o “como” dessas intenções. O Cython converte dataclasses ou DTOs Python em mensagens nativas, cuida da tradução de exceções e evita que detalhes de STL, templates ou layout binário vazem para cima.

4.2. Componentes recomendados

No control plane, os componentes mínimos são um *API Service*, um *Runtime Orchestrator*, um *State Reconciler*, um *Scenario Manager*, um *Metadata Repository* e um *Artifact Index*. O *API Service* expõe operações como criar sandbox, iniciar, anexar, sinalizar, parar, inspecionar, criar campanha destrutiva, iniciar run e obter artefatos. O *Runtime Orchestrator* traduz essas chamadas em comandos de domínio e as envia ao data plane. O *State Reconciler* observa a trilha de eventos e converge o estado persistido para o estado do mundo. O *Scenario Manager* conhece os cenários *sleep*, *shell*, *fork storm*, *OOM*, *zombie* e sinais. O repositório de metadados persiste modelos de domínio e o índice de artefatos aponta para transcripts, dumps, métricas e snapshots.

No data plane, os componentes mínimos são um *Runtime Agent*, um *Init Shim*, um *Namespace Launcher*, um *Cgroup Manager*, um *PTY Manager*, um *Signal Router*, um *Reaper*, um *Resource Observer* e um *Cleanup Engine*. O *Runtime Agent* é o processo privilegiado por host. O *Init Shim* é um binário mínimo estático executado como PID 1 dentro do namespace do contêiner. O *Namespace Launcher* prepara o ambiente de isolamento. O *Cgroup Manager* escreve limites, observa contadores e mantém a árvore de cgroups. O *PTY Manager* detém o master e cuida de attach, detach e resize. O *Signal Router* implementa a diferença entre sinais do operador e sinais provenientes da semântica do terminal. O *Reaper* limpa filhos encerrados. O *Resource Observer* transforma sinais do kernel em eventos de domínio. O *Cleanup Engine* garante teardown idempotente, inclusive em cenários em que o workload está resistindo.

Figura 1. Arquitetura recomendada para a feature

O Python governa intenção, estado e auditoria; o C++ governa syscalls, FDs e supervisão; o rt-init garante governabilidade dentro do PID namespace.

Figura 1. Arquitetura recomendada para a feature. O Python governa intenção, estado e auditoria; o C++ governa syscalls, FDs e supervisão; o rt-init garante governabilidade dentro do PID namespace.

4.3. Modelo de domínio e DDD

A linguagem ubíqua proposta para o projeto deve separar “Sandbox” de “Container Image” e de “ExecSession”. *Sandbox* é o agregado raiz do ambiente de execução; ele possui um init, um conjunto de limites de recurso, um estado de ciclo de vida e zero ou mais sessões de execução secundária. *ExecSession* representa uma entrada adicional no mesmo sandbox, geralmente obtida por *setns*, que herda restrições e aumenta a complexidade de reaping. *ResourceEnvelope* agrupa políticas de memória, PIDs, CPU, swap, OOM de grupo e tempos de grace period. *TerminationIntent* representa a intenção do operador ao parar o sandbox. *ScenarioRun* representa a execução de um cenário destrutivo com expectativas explícitas. *Artifact* referencia um transcript, um snapshot de */proc*, um dump de eventos ou um resumo de métricas.

Os bounded contexts naturais são cinco. O contexto de *Bundle and Image Assembly* cuida de rootfs e de *config.json*. O contexto de *Runtime Orchestration* cuida de ciclo de vida e políticas operacionais. O contexto de *Execution and Isolation* vive no data plane e encapsula namespaces, cgroups, PTY, sinais e processos. O contexto de *Destructive Verification* modela cenários, expectativas, asserções e campanhas. O contexto de *Observability and Incident Analysis* consolida eventos, timelines e artefatos. Separar esses contextos evita que a lógica de “como criar um PTY” invada a lógica de “qual requisito um cenário deve comprovar”.

Há também invariantes úteis de domínio. Um sandbox deve ter exatamente um init ativo em qualquer instante. Todo *ScenarioRun* deve possuir um cgroup pai de guarda para limitar blast radius e um cgroup filho dedicado ao workload. Uma execução só pode ser

considerada realmente encerrada quando a subárvore de cgroups estiver vazia, os mount points tiverem sido limpos e nenhum FD que represente PTY ou pipes de I/O permanecer aberto por engano. Essas invariantes não são ornamentais; são justamente o que evita vazamentos invisíveis que, meses depois, aparecem como “o host ficou estranho”.

5. Análise aprofundada de cada conceito

5.1. Sleep

Sleep é o caso mais simples e, por isso mesmo, um dos mais valiosos. Em termos operacionais, ele representa um processo que não faz quase nada além de permanecer vivo por muito tempo. Em termos de teste, ele exerce o caminho mínimo do runtime: criação de sandbox, persistência de metadados, montagem do ambiente, lançamento do processo, observação de liveness, parada por sinal, coleta de código de saída e cleanup. No Docker do dia a dia, comandos como *sleep 600* ou *top* em foreground são usados como sentinelas para depuração, attach e demonstração de ciclo de vida [7][9]. O fato de esse caso ser “entediante” é exatamente o que o torna útil como canário.

No Docker original, um contêiner rodando *sleep* evidencia imediatamente se a semântica de parada está correta. Se o processo principal for de fato o alvo apropriado do stop signal, o contêiner deve encerrar rapidamente sob SIGTERM ou sob o sinal configurado; se houver wrappers incorretos, shell-form em ENTRYPOINT ou problemas de PID 1, o runtime pode acabar precisando escalar para SIGKILL [9][10]. Essa observação é diretamente aplicável ao projeto. O cenário *sleep* deve existir não apenas como “smoke test”, mas como pré-condição para todos os demais cenários. Se *sleep* não funciona com semântica limpa de start, observe e stop, qualquer sucesso posterior em cenários mais caóticos será enganoso.

A implementação recomendada é usar um helper explícito, e não depender exclusivamente de um binário de distribuição. Um pequeno executável de teste pode aceitar parâmetros como duração, reação a sinais, escrita opcional em stdout e comportamento de foreground ou background. Isso dá ao control plane um alvo determinístico e reduz variabilidade entre imagens. O sandbox para *sleep* deve registrar pelo menos: timestamp monotônico de *spawn*, confirmação de estado *running*, PID do init, PID do filho principal, estado do cgroup, código de saída, sinal terminal, duração até parada e latência de cleanup. Um bom run de *sleep* serve como linha de base para detectar regressões de latência no supervisor, vazamentos de FD, erros de state machine e perda de observabilidade.

O valor arquitetural de *sleep* aparece sobretudo na reconciliação. É o cenário ideal para validar se o estado observado do data plane chega ao Python com ordem causal inteligível. Se o runtime agent emite “started”, “running”, “signal sent”, “exited” e “cleaned” fora de ordem ou com relógios inconsistentes, o problema aparecerá aqui de modo nítido. Por isso, o cenário de *sleep* deve ser usado também como teste de integridade da trilha de eventos. Em um sistema maduro, ele se torna o equivalente do “hello world” da confiabilidade do runtime.

5.2. Shell interativa

A shell interativa é o ponto em que o runtime deixa de lidar apenas com processo e passa a lidar com terminal. Um shell não precisa só de STDIN, STDOUT e STDERR; ele precisa de um pseudo-TTY, de controle de eco, de modo canônico, de `resize`, de foreground process group e, muitas vezes, de um canal de detach que não mate o contêiner. O Docker modela essa necessidade por meio das flags `-i` e `-t`, deixando explícito que `-i` mantém STDIN aberto e `-t` aloca um pseudo-TTY [7]. A documentação mostra inclusive o efeito visível de TTY sobre programas como *passwd*, nos quais a ocultação de caracteres depende do terminal [7]. Essa observação é central: shell interativa não é só “streaming de bytes”, é semântica terminal.

No projeto proposto, o dono do PTY deve ser o data plane em C++. O motivo é simples: o PTY master é um recurso de kernel de longa duração, sensível a fechamento acidental, `resize`, back-pressure e multiplexação. Se ele ficasse subordinado ao processo HTTP ou a um event loop Python não especializado, seria fácil produzir estados estranhos, principalmente sob attach/detach ou sob clientes lentos. O *PTY Manager* no C++ deve abrir o par com *openpty()* [19], configurar o slave para o processo filho, tornar o filho líder de sessão e controlador do terminal, e manter o master aberto enquanto o sandbox existir. O control plane em Python recebe apenas uma abstração de sessão, não o recurso cru.

A decisão mais sutil diz respeito a sinais interativos. Há dois caminhos diferentes. O primeiro é o caminho do operador: um *stop* ou *kill* emitido pelo control plane deve ser traduzido em sinal explícito até o init do contêiner. O segundo é o caminho do terminal: CTRL-C, CTRL-Z e `resize` nascem da interação do cliente com a sessão e, idealmente, devem seguir o comportamento tradicional do terminal. Em uma sessão TTY, o mais correto é que o cliente envie bytes e eventos de `resize` ao PTY, deixando que a line discipline do kernel produza SIGINT ou SIGWINCH no grupo foreground dentro do namespace. Se, além disso, o runtime ainda injetar um SIGINT por fora, o resultado pode ser duplicidade de sinal e bugs difíceis de reproduzir. Essa é uma distinção operacional que o sistema precisa explicitar.

Os testes desse cenário devem ir além do simples “o prompt apareceu”. Eles devem cobrir composição bidirecional de I/O, preservação de modos do terminal do cliente, detach sem encerramento do contêiner, reattach sem corrupção do fluxo, `resize` de janela com propagação de SIGWINCH, comportamento de aplicativos sensíveis a TTY e persistência de transcript. Também é importante exercitar carga de saída alta, porque a documentação do *docker attach* lembra que buffers de attach podem introduzir back-pressure e afetar o ritmo da aplicação [8]. Mesmo que o projeto escolha outra implementação de buffering, o princípio continua válido: shell interativa é um problema de fluxo e latência, não apenas de alocação inicial de TTY.

Sob a ótica de DDD, uma shell interativa não deve ser modelada como atributo incidental do contêiner. Ela é uma *ExecSession* ou uma *PrimaryInteractiveSession*, com identidade própria, política de detach, política de sinais, transcript, dimensão da janela e vínculo temporal com um sandbox. Essa modelagem permite que o produto exponha uma API coerente, inclusive para observabilidade e auditoria. Também ajuda a evitar um erro comum: acoplar demais o protocolo de rede do cliente à semântica do terminal. O control plane deve dizer “redimensione para 132x43”, e o data plane deve saber o que isso significa em termos de ioctl.

5.3. Fork bomb

A fork bomb é um caso clássico de comportamento malicioso ou acidentalmente patológico. Sua essência é consumir a capacidade de criação de processos mais rápido do que o sistema consegue reciclar PIDs ou impor limites. Em shell, a forma clássica é conhecida e infame; em um runtime de produção, porém, usar a bomba “pura” é desnecessariamente arriscado. Para este projeto, a recomendação é substituir a bomba clássica por um helper controlado que execute um laço de *fork/clone* até encontrar falha previsível, registrando o erro retornado, a quantidade de filhos e o tempo de reação do supervisor. O objetivo não é destruir o host; é provar contenção, observabilidade e capacidade de limpeza.

No Docker, a superfície pública desse problema aparece em *--pids-limit* [7]. No kernel, a documentação de cgroup v2 define *pids.max* como limite duro do número de processos e *pids.events* como contador de vezes em que o grupo bateu esse teto [12]. A mesma documentação ressalta uma nuance importante: operações organizacionais podem fazer *pids.current* ficar maior que *pids.max*, mas não é possível violar a política por meio de *fork()* ou *clone()*, que retornam erro quando a criação excederia o limite [12]. Isso significa que uma asserção correta para o cenário não deve ser “*pids.current* nunca ultrapassou o limite”, e sim “o limite conteve a criação e o runtime permaneceu controlável”.

A melhor forma de isolar o teste é hierárquica. Cada run destrutivo deve nascer sob um cgroup pai de guarda, com orçamento próprio de memória, CPU e PIDs, e sob um cgroup filho específico do sandbox. O pai protege o host do caso em que a limpeza do filho falha; o filho representa o contêiner. Em caso de anomalia severa, o *Cleanup Engine* pode acionar *cgroup.kill* no pai ou no filho para extinguir toda a subárvore por SIGKILL [12]. Essa estratégia é muito mais robusta do que tentar matar apenas o processo líder, porque uma fork bomb frequentemente cria uma floresta de descendentes cuja topologia muda enquanto o operador reage.

A relação entre fork bomb e zombie reaping também é profunda. Uma tempestade de processos só fica realmente sob controle se o sistema sabe reapear rapidamente os que morrem. Se houver atraso ou perda no caminho de SIGCHLD, ou se o init do namespace não reapeia adequadamente, o teste deixa de medir apenas contenção e passa a revelar vazamento de zombies. Isso não é um efeito colateral; é uma das razões pelas quais o cenário é tão valioso. Um runtime maduro deve conseguir responder a três perguntas depois desse teste: quantas tentativas de criação foram bloqueadas, quantos descendentes precisaram ser reapados e quanto tempo levou até o cgroup voltar ao basal.

Do ponto de vista do desenho de produto, este cenário pede uma política de segurança explícita. O *ScenarioRun* de fork storm deve declarar que roda apenas em hosts de teste ou em VMs efêmeras, que exige limites de PIDs obrigatórios, que define timeout curto de grace period e que admite limpeza forçada automática. Essa formalização impede que a feature seja usada de forma irresponsável em ambientes compartilhados. Ela também torna o sistema auditável: a própria plataforma sabe que aquele run era destrutivo e, portanto, deve deixar rastro maior de telemetria e isolamento.

5.4. OOM

OOM é um termo que parece simples, mas encobre semânticas diferentes. Um workload pode cruzar um limite de memória e sofrer throttling; pode ter uma alocação negada com *ENOMEM*; pode fazer o kernel invocar o OOM killer dentro do cgroup; ou pode arrastar o host para uma condição global ruim caso a política de limites esteja errada. A documentação de recursos do Docker é enfática em dois pontos: não se deve permitir que um contêiner consuma memória do host sem controle, e não é recomendável contornar as proteções padrão com ajustes agressivos de OOM score ou com uso irresponsável de *--oom-kill-disable* [11]. Em um runtime do zero, isso significa que o cenário de OOM não pode ser tratado como simples “deixe o processo morrer”.

O kernel fornece um modelo mais rico do que um booleano “OOMKilled”. Em cgroup v2, *memory.high* é um limite de pressão que desacelera o grupo e nunca, por si só, invoca o OOM killer; já *memory.max* é o limite duro que pode levar ao estado de OOM [12]. A telemetria de *memory.events* informa quantas vezes o grupo entrou em pressão alta, bateu o máximo, entrou em situação de OOM e matou processos [12]. Além disso, *memory.oom.group* permite tratar o grupo como unidade indivisível, evitando mortes parciais que deixam o workload em estado semivivo [12]. Essa diferença é valiosa para testes: uma suíte bem desenhada consegue observar pressão antes de observar morte.

A implementação recomendada é um helper de memória deliberadamente determinístico: ele aloca páginas anônimas, toca a memória para garantir commitment e pode subir em degraus definidos. O *ResourceEnvelope* do cenário deve preferencialmente configurar um *memory.high* abaixo de *memory.max*, para registrar o momento em que a carga entrou em pressão, e então um teto duro para validar a reação terminal. Em ambientes onde se deseja máxima determinismo, o swap do cenário deve ser limitado ou desabilitado, para evitar que o run se transforme apenas em thrash. O init e o runtime agent devem registrar não só o código de saída final, mas também o histórico de eventos de memória, os picos de uso e a causa classificada da falha.

Uma nuance importante da própria documentação do kernel é que nem toda alocação que encontra o limite invoca o OOM killer; algumas podem falhar de maneiras diferentes, retornar *-ENOMEM* ou ser tratadas pelo chamador [12]. Logo, uma asserção madura não deve depender exclusivamente de “o processo morreu”. O cenário pode ser considerado bem-sucedido se o runtime detectar corretamente que o workload excedeu seu envelope e registrar, com precisão, se isso resultou em throttling, falha de alocação, OOM local ou kill. Essa diferença importa porque, em produção, o operador quer saber se a aplicação saiu por comportamento próprio ou por política de contenção do runtime.

Do ponto de vista arquitetural, OOM é também um teste de observabilidade. Um runtime que só devolve “exit code 137” e nada mais falha como produto. O control plane deve persistir pelo menos: uso corrente e pico de memória, contadores de *memory.events*, política de swap, configuração de *memory.oom.group*, timestamp do primeiro sinal de pressão, timestamp do evento terminal e classificação final da causa. Essa riqueza permite construir uma experiência parecida com o que usuários esperam de runtimes maduros, sem depender de heurísticas frágeis sobre códigos de saída.

5.5. Zombie reaping

Zombie reaping é menos visível para quem observa apenas a aplicação, mas absolutamente central para o runtime. Um processo zombie já terminou sua execução; ele continua existindo apenas porque seu pai ainda não coletou seu estado. Em escala pequena, isso parece inofensivo. Em escala de runtime, especialmente sob fork storms ou sob workloads que fazem *double-fork*, zombies consomem slots de PID, poluem a árvore de processos e quebram a previsibilidade do supervisor. O Linux torna o problema ainda mais importante dentro de PID namespaces, porque o processo PID 1 do namespace se torna pai de órfãos [13].

O Docker reconhece explicitamente essa questão ao oferecer *--init* e ao documentar que um init dentro do contêiner garante responsabilidades usuais como reap de zombies [7]. A implementação padrão do *docker-init* é baseada em *tini*, cuja própria documentação se apresenta como “um init mínimo, porém válido”, responsável por reapear zombies e encaminhar sinais [20]. Essa escolha do ecossistema não é casual. Ela condensa anos de bugs de produção em uma decisão simples: o processo principal de um contêiner não deve ser presumido como um bom PID 1.

Para o projeto em Python/Cython/C++, a conclusão é forte: o recurso deve nascer com um *init shim* mínimo, mesmo que o restante do produto ainda esteja em fase inicial. Esse shim roda como PID 1 do namespace do contêiner, bloqueia sinais relevantes, abre um *signalfd*, lança o workload principal como líder de grupo de processo, reapeia filhos encerrados e propaga o encerramento final do workload de volta ao runtime agent. Em paralelo, o shim externo do host deve marcar-se como *subreaper* com *PR_SET_CHILD_SUBREAPER* [14], porque ainda haverá processos auxiliares, exec sessions e caminhos em que o reparenting relevante ocorre fora do namespace principal.

O cenário destrutivo específico de zombie reaping deve usar helpers que criam filhos que saem sem ser esperados, filhos que fazem *double-fork* e, idealmente, exec sessions que entram no namespace já existente. Isso permite validar a observação do *containerd* Runtime v2 de que, em novos PID namespaces, o init interno precisa assumir limpeza que o shim externo não consegue garantir por si só [3]. A asserção correta não é apenas “nenhum zombie existe agora”, mas “zombies transitórios são reapeados dentro de um limiar e não se acumulam”. Em termos de produto, esse limiar deve ser mensurável e exposto em artefatos.

Esse cenário também afeta a semântica de desligamento. Se o runtime decide encerrar um contêiner que tem filhos ainda vivos, o init interno deve continuar desempenhando seu papel até que a árvore seja realmente encerrada. Um erro comum em runtimes jovens é tratar a morte do processo principal como suficiente, desanexando cedo demais os mecanismos de espera. Isso produz justamente os problemas mais irritantes: processos “fantasmas”, cgroups que não esvaziam e teardown que falha intermitentemente.

5.6. Signal forwarding

Signal forwarding é o mecanismo pelo qual intenções do operador, do cliente interativo ou do próprio sistema operacional chegam ao workload de forma previsível. À primeira vista, parece ser apenas “dar kill no PID certo”. Na prática, é um dos contratos mais frágeis do runtime. O *docker stop* define um comportamento explícito: envia o sinal configurado de parada e, depois do timeout, força SIGKILL [9]. O *docker attach* documenta *--sig-proxy* e lembra que o PID 1 do contêiner ignora diversos sinais de ação padrão caso não instale handlers apropriados [8]. O Dockerfile reference mostra como um shell wrapper pode desviar essa semântica [10]. Logo, signal forwarding não é apenas uma syscall; é uma política de entrega.

O desenho recomendado separa três níveis de encaminhamento. O primeiro é o nível do operador: o control plane emite uma intenção de parada, interrupção, hangup, reload ou kill. O runtime agent em C++ recebe essa intenção e, no host, entrega o sinal ao PID 1 do sandbox por meio de *pidfd_send_signal()* sempre que possível [16]. O segundo é o nível do init interno: ao receber o sinal, o *rt-init* decide como propagá-lo ao workload, tipicamente para o grupo de processos do filho principal, e não apenas para um PID isolado, para evitar sobras. O terceiro é o nível do terminal: sinais gerados por CTRL-C, CTRL-Z e resize em sessões TTY devem preferir a semântica natural do PTY e do foreground process group. Misturar esses níveis é a receita para bugs intermitentes.

Há um equilíbrio delicado entre compatibilidade e sanidade operacional. Encaminhar apenas para o PID líder preserva semânticas de aplicações que gerenciam seus próprios filhos; encaminhar para o grupo inteiro melhora a probabilidade de teardown completo em shells, pipelines e workloads com subprocessos cooperativos. A solução recomendada é modelar essa escolha como uma *KillEscalationPolicy* e como uma *SignalFanoutStrategy*, em vez de enterrá-la como detalhe hard-coded. O padrão de projeto adequado aqui é *Strategy*: para *sleep* e OOM, o alvo pode ser o filho principal; para shell interativa e fork storm, o grupo de processo costuma ser mais seguro. O importante é que o comportamento seja explícito no metamodelo e auditável no evento final.

As asserções do cenário devem cobrir pelo menos SIGTERM, SIGINT, SIGHUP, SIGQUIT, SIGWINCH e SIGKILL. O sistema deve registrar o sinal solicitado, o instante de envio, o alvo lógico, o alvo físico, a resposta do workload, o instante de escalonamento e o resultado final. Também deve garantir ausência de duplicidade. Um contêiner que recebe SIGINT duas vezes — uma pelo PTY e outra pelo supervisor externo — pode se comportar de forma não determinística. O mesmo vale para resize: uma sessão responsiva deve refletir dimensões novas com rapidez, mas isso não deve ser confundido com “um sinal do operador”. A modelagem correta evita que toda forma de controle seja comprimida em um único botão conceitual chamado “signal”.

6. Plano de implementação em baixo nível

A passagem do desenho arquitetural para uma implementação concreta exige um princípio de disciplina: todo trecho que interage diretamente com o kernel deve viver atrás de um contrato estreito, explícito e versionado. Esse princípio é particularmente importante em um projeto híbrido. Se o Python puder abrir arbitrariamente descritores, manipular PIDs “na mão” e escrever em arquivos de cgroup fora do fluxo normal do runtime agent, a fronteira entre control plane e data plane deixa de existir na prática. O resultado costuma ser um sistema difícil de reconciliar, porque parte do estado real do host passa a nascer fora do caminho observado. A proposta, portanto, é fazer do C++ o único lugar autorizado a executar operações privilegiadas e a manter handles sensíveis, enquanto o Python trabalha exclusivamente com intenções, modelos e evidências. O Cython entra como a camada que traduz estruturas de domínio em chamadas tipadas, liberando o GIL durante operações bloqueantes e convertendo erros nativos em exceções semanticamente úteis para o domínio [15][16][17][21][22][23].

6.1. Contratos entre Python, Cython e C++

O contrato mais simples e mais seguro é um conjunto de comandos idempotentes e um fluxo de eventos ordenado. No sentido de ida, o Python envia pedidos como *CreateSandbox*, *StartSandbox*, *AttachSession*, *ResizePty*, *SendSignal*, *StopSandbox*, *KillSandbox*, *RunScenario* e *CollectArtifacts*. No sentido de volta, o data plane publica *SandboxPrepared*, *InitStarted*, *WorkloadStarted*, *PressureObserved*, *SignalDelivered*, *ExitObserved*, *CgroupKilled*, *CleanupCompleted* e falhas estruturadas. O ponto decisivo é que o contrato deve trafegar identificadores lógicos do domínio e nunca apenas inteiros de PID. O PID pode existir como detalhe interno ou como artefato de diagnóstico; a API de produto precisa falar em *SandboxId*, *ExecSessionId*, *ScenarioRunId* e *TerminationIntent*.

A recomendação prática para o Cython é expor uma fachada mínima, não uma coleção difusa de bindings. Em vez de envolver dezenas de classes C++ e STL containers no Python, o projeto deve publicar poucas operações bem modeladas, idealmente sobre DTOs estáveis e simples. A própria documentação do Cython favorece esse desenho ao permitir declarar headers externos, classes C++ e traduções de exceção de forma controlada [21][22][23]. Isso reduz o risco de vazamento de ABI, facilita testes de contrato e permite que o data plane evolua internamente sem obrigar o control plane a conhecer detalhes de alocadores, templates ou ownership de FDs.

```
# cython: language_level=3
# Exemplo ilustrativo de uma fachada estreita entre Python e C++

cdef extern from "runtime_api.h" namespace "rt":
    cdef cppclass RuntimeError(Exception):
        pass

    cdef cppclass StartRequest:
        string sandbox_id
        string bundle_path
        bint terminal
        string cgroup_path
        string stop_signal
        int grace_timeout_ms

    cdef cppclass StartReply:
        int init_pid
        int pidfd
        string state

    StartReply start_sandbox(StartRequest&) except +
    void send_signal(string sandbox_id, int signo) except +
    void resize_pty(string sandbox_id, int rows, int cols) except +
    void collect_artifacts(string sandbox_id, string output_dir) except +
```

```

cdef class RuntimeFacade:
    def start(self, sandbox_id: str, bundle_path: str, terminal: bool):
        cdef StartRequest req
        req.sandbox_id = sandbox_id.encode()
        req.bundle_path = bundle_path.encode()
        req.terminal = terminal
        req.grace_timeout_ms = 10000
        with nogil:
            reply = start_sandbox(req)
        return {"init_pid": reply.init_pid, "state": reply.state.decode()}

```

Esse exemplo resume duas ideias importantes. A primeira é que o Python trabalha com uma interface orientada a intenção, e não com detalhes de *clone3()*, *setns()* ou *epoll*. A segunda é que chamadas potencialmente bloqueantes precisam liberar o GIL. Um runtime que segura o GIL enquanto espera um evento de processo, um resize de PTY ou uma resposta do agente introduz acoplamento desnecessário entre tráfego de API e mecanismos do kernel. Em uma carga destrutiva, isso rapidamente se torna um gargalo e uma fonte de deadlock operacional.

6.2. Runtime agent, init shim e laço de supervisão

O *Runtime Agent* deve ser um processo por host. Sua responsabilidade é preparar cgroups, montar o bundle, abrir sockets e PTYs, lançar o init interno e conduzir a limpeza. Ele pode ser implementado como um processo persistente, com um laço *epoll*-centrado que multiplexa sockets de comando, *pidfds*, *signalfd*, pipes de stderr/stdout, descritores de PTY e notificações de observadores de recursos. Esse agente deve registrar-se como *subreaper* do lado de fora com *PR_SET_CHILD_SUBREAPER* [14], mas não deve presumir que isso elimina a necessidade de um init interno. Em novos PID namespaces, o primeiro processo continua sendo semanticamente especial, adota órfãos e condiciona a própria sobrevivência do namespace [13]. É por isso que o projeto precisa de duas camadas de supervisão: uma host-side e outra namespace-side.

O *Init Shim* interno, chamado aqui de *rt-init*, deve ser deliberadamente pequeno. Sua missão não é orquestrar o produto; sua missão é tornar o sandbox governável. Ele entra como PID 1 do namespace, prepara o grupo de processos do workload, reapeia qualquer filho que finalize, encaminha sinais segundo uma política explícita e devolve ao agente um resultado terminal inequívoco. Em produção, a melhor forma de mantê-lo pequeno é resistir à tentação de colocar nele lógica de negócio, parsing complexo ou integrações que pertencem ao control plane. Ele deve se parecer muito mais com um *tini* especializado do que com um mini-daemon geral [20].

```

// Esboço conceitual de rt-init em C++
int main(int argc, char** argv) {
    sigset_t mask;
    sigemptyset(&mask);
    sigaddset(&mask, SIGCHLD);
    sigaddset(&mask, SIGTERM);
    sigaddset(&mask, SIGINT);
    sigaddset(&mask, SIGHUP);
    sigaddset(&mask, SIGQUIT);
    sigprocmask(SIG_BLOCK, &mask, nullptr);

    int sfd = signalfd(-1, &mask, SFD_CLOEXEC);
    pid_t child = fork();

    if (child == 0) {
        setsid(); // nova sessão
        setpgid(0, 0); // novo grupo de processo
        // opcional: associar slave PTY como controlling terminal
        execve(workload_path, workload_argv, workload_envp);
        _exit(127);
    }

    int child_pfd = pfd_open(child, 0);
    int ep = epoll_create1(EPOLLCLOEXEC);
    add_fd(ep, sfd);
}

```

```

add_fd(ep, child_pidfd);

for (;;) {
    auto ev = wait_one(ep);

    if (ev.fd == sfd) {
        auto info = read_signalfd(sfd);
        if (info.ssi_signo == SIGCHLD) {
            reap_all_children_nonblocking(); // waitid/waitpid em loop
            if (child_has_exited(child)) {
                return map_child_status(child);
            }
        } else {
            // Política: encaminhar ao grupo do workload
            kill(-child, info.ssi_signo);
        }
    }

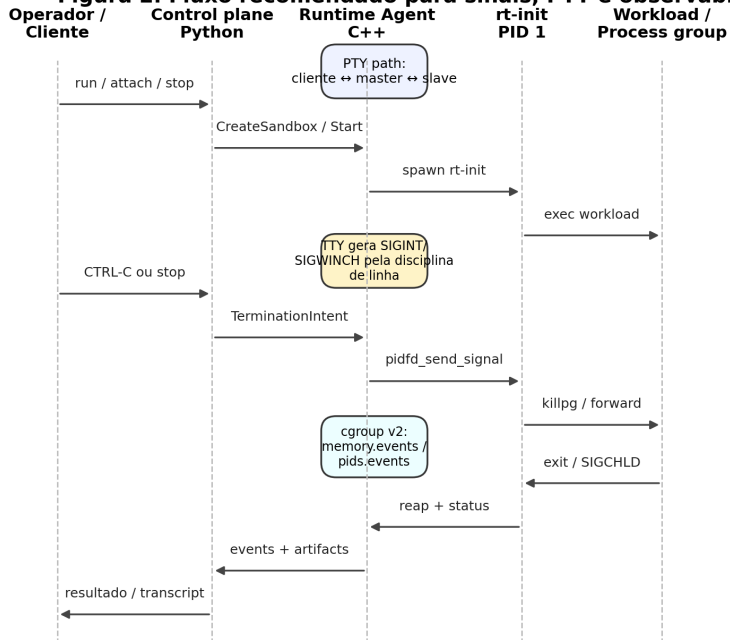
    if (ev.fd == child_pidfd && child_pidfd_is_ready(child_pidfd)) {
        reap_all_children_nonblocking();
        return map_child_status(child);
    }
}
}

```

O esboço acima omite detalhes relevantes de produção, mas captura o centro da ideia. Em vez de espalhar handlers assíncronos por todo o processo, o *init* transforma sinais em eventos de FD por meio de *signalfd()* [17]. Em vez de confiar em PIDs nus, monitora o processo principal com *pidfd_open()* e o sinaliza por *pidfd_send_signal()* quando a entrega precisa ser feita do lado host [15][16]. Em vez de assumir que apenas o processo principal importa, reapeia todos os filhos até esgotar a fila. E, sobretudo, encapsula a regra de que o “fim do contêiner” não é um único *waitpid*; é a combinação entre saída do workload, exaustão da árvore observável e limpeza coerente da subárvore de cgroup.

6.3. PTY, sessões interativas e semântica de terminal

O gerenciamento de terminal merece uma trilha própria de implementação. O *PTY Manager* deve usar *openpty()* para obter o par master/slave [19]. O slave é passado ao workload dentro do sandbox; o master permanece com o runtime agent, que o multiplexa para os clientes anexados. Se a sessão for a principal do contêiner, o *init* ou o workload precisa receber o terminal como controlling terminal da nova sessão, respeitando as convenções usuais de *setsid()*, foreground process group e tamanho inicial de janela. O controle de *resize* deve acontecer por *ioctl(TIOCSWINSZ)* no lado slave, seguido da propagação natural de *SIGWINCH* para os processos interessados. Isso é preferível a “simular” *resize* por meio de sinais enviados de fora, porque preserva o comportamento terminal clássico descrito pela própria semântica de PTY [18][19].

Figura 2. Fluxo recomendado para sinais, PTY e observabilidade

O caminho de operador usa intenção → agente → pidfd → init; o caminho interativo usa PTY e a disciplina de linha do terminal. Ambos convergem em eventos auditáveis.

Figura 2. Fluxo recomendado para sinais, PTY e observabilidade. O caminho de operador usa intenção → agente → pidfd → init; o caminho interativo usa PTY e a disciplina de linha do terminal. Ambos convergem em eventos auditáveis.

A sessão interativa também precisa de uma política formal de attach e detach. O attach pode ser efêmero; o sandbox não. Portanto, desconectar um cliente não deve fechar o master PTY enquanto houver sessão viva. A identidade do terminal pertence ao sandbox ou à *ExecSession*, não ao socket TCP ou websocket específico que o operador estava usando naquele instante. Essa distinção é importante para evitar o erro comum em runtimes jovens: confundir fim de conexão de rede com fim de execução do workload. O controle de back-pressure deve ser explícito, com buffers limitados e política de descarte ou bloqueio bem definida, para que um cliente lento não torne o processo supervisor indisponível.

6.4. Observabilidade, artefatos e reconciliação

Como a própria feature é destrutiva, a qualidade do produto dependerá fortemente da qualidade das evidências. Cada *ScenarioRun* deve persistir uma trilha temporal minimamente completa: bundle usado, hash do helper, parâmetros do cenário, configuração final de namespaces, configuração final de cgroups, valores de *memory.current*, *memory.peak*, *memory.events*, *pids.current*, *pids.peak* e *pids.events*, timeline de sinais, transcript de PTY quando aplicável, stderr do init, stdout/stderr do workload, causa terminal classificada e resultado da limpeza. A própria documentação do cgroup v2 faz de *memory.events* e *pids.events* fontes privilegiadas para saber se o grupo bateu pressão ou saturou o teto de processos [12]. Um runtime que ignora esses arquivos perde a chance de transformar um incidente destrutivo em observabilidade acionável.

A reconciliação no control plane deve partir do princípio de que o agente pode falhar no pior instante possível. Por isso, o estado persistido não deve depender apenas de mensagens transitórias. Depois de qualquer reinício do API Service ou do Runtime Orchestrator, o sistema precisa varrer sandboxes conhecidos, reabrir contexto do agente

quando possível, confirmar existência de subárvores de cgroup, verificar se o init ainda está vivo por meio de *pidfd* ou, na sua ausência, por validação de processo, e concluir runs órfãos de modo determinístico. Esse modelo aproxima o projeto do que o ecossistema Docker/containerd aprendeu ao longo do tempo: supervisão durável não é apenas iniciar processos, mas sobreviver a quedas parciais de componentes sem perder autoridade sobre o estado [1][2][3].

Tabela 1. Cobertura dos cenários destrutivos e subsistemas implicados

Cenário	Subsistemas exercitados	Evidências mínimas esperadas
Sleep	Lifecycle básico, OCI bundle, init, stop semantics, reconciliação e limpeza	Linha do tempo start→running→signal→exit→cleanup, latência de parada, ausência de FDs e cgroups residuais
Shell interativa	PTY manager, attach/detach, resize, foreground process group, transcript	Prompt responsivo, reattach sem corrupção, resize efetivo, detach sem matar o sandbox, transcript íntegro
Fork bomb	pids controller, guard cgroup, reaping, limpeza forçada, responsividade do agente	Incremento de pids.events.max, rejeição de clone/fork além do teto, agente vivo, retorno de pids.current ao baseline
OOM	memory.high, memory.max, memory.events, classificação de causa, cleanup	Registro de pressão, registro de oom/oom_kill quando ocorrer, preservação do host, classificação terminal inteligível
Zombie reaping	init interno, subreaper externo, wait loops, teardown tardio	Ausência de zombies persistentes, árvore drenada antes do estado cleaned, teardown consistente
Signal forwarding	pidfd, signalfd, políticas de fanout, stop/kill escalation, integração com TTY	Entrega correta de TERM/INT/HUP/QUIT/WINCH/KILL, ausência de duplicidade, timeline de escalonamento completa

7. System Design, DDD e padrões de projeto aplicados

Do ponto de vista de *System Design*, a decisão principal já apareceu: separar o sistema em um plano de controle semanticamente rico e um plano de execução operacionalmente duro. Mas isso não basta. O desenho precisa de uma topologia de responsabilidade que resista a falhas. O control plane não deve ser a fonte única da verdade sobre a vida imediata do processo; essa verdade momentânea mora no agente. Ao mesmo tempo, o agente não deve ser a fonte única da verdade sobre intenção, política, histórico e reconciliação; isso mora no control plane. O sistema correto é aquele em que cada lado possui a verdade adequada ao seu domínio e ambos convergem por eventos, snapshots e idempotência. Em termos práticos, isso significa que a API pode cair sem destruir um shell em execução, e o agente pode reiniciar sem tornar a API incapaz de explicar o que ocorreu.

Em termos de *Low Level Design*, o projeto deve preferir objetos pequenos, ownership explícito e semântica de fechamento determinística. Em C++, cada FD, mount namespace handle, pidfd, signalfd e descriptor de PTY deve ser encapsulado em RAII. Cada transição de estado deve ser explicitada por tipos e não apenas por convenção verbal. Cada caminho de erro precisa conservar contexto suficiente para classificação no Python. No control plane, o inverso também é verdadeiro: repositórios precisam serializar estados de domínio e não instantâneos crus do kernel; serviços de aplicação precisam coordenar casos de uso e não carregar a mecânica da execução. Essa assimetria é saudável, porque permite que o modelo de domínio permaneça compreensível enquanto o motor operacional evolui.

Sob a ótica de *Domain-Driven Design*, a feature fica mais clara quando se assume que “destruir controladamente” é uma capacidade de domínio. *ScenarioDefinition* não é um script. É um artefato versionado de negócio técnico, com política de segurança, helper associado, envelope de recursos, expectativas e estratégia de classificação. *ScenarioRun* é uma entidade com identidade própria, sujeita a auditoria e repetição. *TerminationIntent* é um Value Object que expressa causa esperada e política de escalonamento. *ResourceEnvelope* é outro Value Object que descreve limites e guardrails. Essa linguagem ubíqua evita que o sistema se degrade em uma coleção de flags soltas.

Os padrões GoF entram como estabilizadores da forma. O *Facade* aparece naturalmente na fronteira Cython, expondo uma API pequena sobre um interior C++ mais rico. O *Builder* é útil para compor bundles OCI e descrições de cenário sem construtores telescópicos. O padrão *State* é quase obrigatório para lifecycle de sandbox e lifecycle de *ScenarioRun*, porque os comportamentos válidos mudam fortemente entre *Prepared*, *Running*, *Stopping*, *Exited* e *Cleaned*. O padrão *Strategy* resolve a variabilidade legítima de fanout de sinais, de escalonamento de kill, de política de reaping e de classificação terminal. O padrão *Observer* organiza o fluxo de eventos do agente para o control plane. O padrão *Command* ajuda a representar operações como start, stop, kill, attach e collect artifacts de forma uniforme e auditável. O padrão *Adapter* aparece na tradução entre DTOs Python e estruturas nativas do runtime. E o padrão *Template Method* é particularmente útil para a suíte destrutiva, porque quase todos os cenários seguem o mesmo arco: preparar, iniciar, estressar, observar, classificar, limpar e publicar artefatos.

Vale notar que alguns dos mecanismos mais importantes do projeto não pertencem estritamente ao catálogo GoF, mas são igualmente decisivos. RAI em C++ é o antídoto natural contra vazamento de descritores. O modelo *Reactor*, materializado por *epoll*, combina bem com *signalfd*, *pidfd* e PTYs, produzindo um laço de eventos coerente. O padrão de *event sourcing* parcial ou, ao menos, de log append-only de eventos do runtime, simplifica auditoria e reconciliação. E o uso de cgroups pai/filho como barreira de blast radius pode ser entendido como um padrão operacional, ainda que não conste nos catálogos clássicos. Em um sistema dessa natureza, design pattern só é útil se reduzir ambiguidade; caso contrário, vira taxonomia ornamental.

8. Requisitos funcionais e não funcionais

Os requisitos abaixo foram formulados para que os seis conceitos do problema sejam cobertos não apenas como testes isolados, mas como capacidades sistemáticas do produto. O objetivo não é definir um backlog exaustivo; é demarcar o mínimo necessário para que a implementação se comporte como runtime confiável e auditável sob estresse.

Tabela 2. Requisitos funcionais recomendados

ID	Descrição
RF-01	O sistema deve criar sandboxes alinhados à semântica OCI, com bundle, configuração de processo, namespaces e envelope de recursos claramente persistidos [4][5][6].
RF-02	O sistema deve iniciar workloads determinísticos de referência para os cenários sleep, shell interativa, fork storm controlada, memhog, geração de zombies e manipulação de sinais.
RF-03	O sistema deve prover um init interno por sandbox, com capacidade de reaping e de encaminhamento de sinais, além de um supervisor externo com papel de subreaper quando aplicável [3][13][14][20].
RF-04	O sistema deve suportar sessões interativas com PTY, incluindo attach, detach, reattach, resize e captura opcional de transcript [7][8][18][19].

ID	Descrição
RF-05	O sistema deve aplicar limites de processos por cgroup, expor métricas e eventos relacionados a saturação de PIDs e classificar falhas decorrentes de fork storms [12].
RF-06	O sistema deve aplicar limites de memória por cgroup, observar pressure e eventos de OOM, e distinguir, na classificação terminal, entre throttling, falha de alocação e morte por OOM killer [11][12].
RF-07	O sistema deve encaminhar sinais de forma configurável, respeitando stop signal, grace period, política de fanout e escalonamento final para SIGKILL ou cgroup.kill quando necessário [8][9][12][16].
RF-08	O sistema deve persistir artefatos operacionais suficientes para auditoria de cada ScenarioRun, incluindo timeline, logs, métricas de cgroup e causa terminal estruturada.
RF-09	O sistema deve expor APIs e/ou CLI para criar ScenarioDefinitions, iniciar ScenarioRuns, inspecionar resultados e coletar artefatos.
RF-10	O processo de cleanup deve ser idempotente e considerar uma execução encerrada apenas quando a árvore de processo, a subárvore de cgroup, os mounts temporários e os FDs críticos tiverem sido drenados.
RF-11	O sistema deve sobreviver à queda do serviço de API sem destruir sandboxes em execução, e deve reconciliar estado após reinício de componentes [1][2][3].
RF-12	O sistema deve permitir política explícita de segurança por cenário, diferenciando runs destrutivos aptos para host de teste, VM efêmera ou ambiente restrito.

Tabela 3. Requisitos não funcionais recomendados

ID	Descrição
RNF-01	Confinamento de blast radius: uma execução destrutiva não pode comprometer a disponibilidade do host além do envelope explicitamente autorizado.
RNF-02	Isolamento de falhas: defeitos no data plane não devem derrubar o serviço de metadados e orquestração, e vice-versa.
RNF-03	Determinismo: helpers, limites e critérios de sucesso devem produzir resultados reproduzíveis entre execuções comparáveis.
RNF-04	Observabilidade: cada run deve gerar evidências suficientes para diagnóstico sem depender de reprodução manual posterior.
RNF-05	Baixa latência de controle: operações de stop, kill, attach e resize devem permanecer responsivas mesmo sob carga patológica moderada.
RNF-06	Segurança: o caminho privilegiado deve ser mínimo, auditável e protegido por política explícita; recursos perigosos devem nascer desabilitados por padrão.
RNF-07	Compatibilidade progressiva: capacidades opcionais do kernel, como pidfd ou cgroup.kill, devem ser detectadas em tempo de inicialização e tratadas por feature flags quando indisponíveis [12][15][16].
RNF-08	Durabilidade: metadados, timeline e artefatos críticos não podem se perder com reinícios normais dos serviços de controle.
RNF-09	Recuperabilidade: o sistema deve reconciliar sandboxes órfãos e runs interrompidos de forma determinística e segura.
RNF-10	Evolução de ABI: a fronteira Cython/C++ deve ser pequena, versionada e testada por contrato para reduzir regressões binárias [21][22][23].
RNF-11	Portabilidade operacional: a primeira versão pode assumir Linux e cgroup v2, mas a base deve explicitar claramente as dependências de kernel e o que ocorre quando elas faltam.
RNF-12	Debugar deve ser barato: o custo de coletar artefatos essenciais deve ser baixo o bastante para permanecer ativado por padrão nos ambientes de teste.

9. Roteiro incremental de implementação

A construção dessa feature fica mais segura quando feita por degraus que preservam valor de diagnóstico desde cedo. A ordem importa porque alguns cenários servem de fundação para os demais. Um runtime que ainda não domina *sleep* e *stop semantics* não deve avançar para fork storm e OOM; um runtime sem init interno coerente não deve tentar provar que lida bem com zombies; um runtime sem PTY sólido produzirá falsos negativos em signal forwarding interativo. A sequência abaixo não é apenas conveniente. Ela reduz risco sistêmico.

Tabela 4. Fases sugeridas de implementação

Fase	Entrega principal	Critério de saída
Fase 0	Capability discovery, checagem de kernel, baseline de cgroup v2, guard cgroup e trilha mínima de eventos.	O host informa claramente quais capacidades estão disponíveis e o cleanup idempotente funciona em runs vazios.
Fase 1	Bundle OCI mínimo, start/stop de sleep, classificação básica de término e reconciliação inicial.	Sleep inicia e encerra repetidamente sem vazamento de FDs, mounts ou cgroups [4][5][9].
Fase 2	Init interno, subreaper externo, timeline de sinais e stop semantics configurável.	TERM e KILL funcionam de modo previsível; zombies simples são drenados; o estado cleaned só aparece após drenagem real [13][14][20].
Fase 3	PTY manager, shell interativa, attach/detach/reattach, resize e transcript.	Sessões interativas sobrevivem a detach; resize chega ao workload; CTRL-C via terminal não duplica sinais [7][8][18][19].
Fase 4	pids controller, helper de fork storm controlada, guardrails e limpeza forçada por cgroup.kill.	Fork storm não compromete o host; pids.events.max registra saturação; cleanup devolve pids.current ao baseline [12].
Fase 5	memory.high, memory.max, memhog determinístico, classificação avançada de OOM e artefatos ricos.	O runtime distingue pressão, ENOMEM e OOM kill, e preserva contexto suficiente para auditoria [11][12].
Fase 6	Endurecimento, políticas de segurança por cenário, testes de queda parcial de componentes e integração contínua destrutiva.	Quedas do control plane e do agente são reconciliadas sem perda de autoridade operacional sobre sandboxes ativos [1][2][3].

Do ponto de vista de risco técnico, a transição mais sensível é a da Fase 2 para a Fase 3. Até esse ponto, o sistema já precisa ter uma semântica robusta de processo e sinais; ao introduzir PTY, ele passa a lidar com outra classe de bugs, ligada a terminal, foreground process groups e fechamento acidental do master. É tentador implementar shell interativa cedo, porque ela “demonstra valor” visualmente. Ainda assim, a ordem sugerida deve ser respeitada. A demo mais bonita não compensa uma fundação errada.

10. Conclusão

Os seis cenários estudados descrevem, juntos, o que significa governar um contêiner de verdade. *Sleep* mede a sanidade do ciclo de vida básico. A shell interativa mede a competência terminal do runtime. A fork bomb mede a contenção de processos e a qualidade da limpeza. O cenário de OOM mede o rigor com que o sistema protege o host e classifica causas. Zombie reaping mede a honestidade do PID 1. Signal forwarding mede a fidelidade do controle operacional. Em um projeto “Docker-like” construído do zero, esses tópicos não pertencem ao final do roadmap; eles são justamente aquilo que distingue um protótipo de um runtime confiável.

A melhor arquitetura para o contexto proposto é aquela em que o Python governa semântica, política, persistência e reconciliação, enquanto o C++ governa syscalls, FDs, cgroups, PTYs, sinais e limpeza. O Cython funciona como cola, mas apenas se a cola for estreita, tipada e disciplinada. O desenho deve alinhar-se à semântica OCI, adotar cgroup v2 como baseline, privilegiar *pidfd*, *signalfd* e RAI1, e assumir desde o começo que um init interno é necessário. Quando isso acontece, a suíte destrutiva deixa de ser um conjunto de scripts perigosos e passa a ser um subsistema de verificação que amadurece o produto.

Se houvesse uma única decisão a reter desta pesquisa, ela seria a seguinte: em runtime de contêineres, confiabilidade não nasce do caminho feliz; ela nasce da capacidade de continuar no comando quando o workload tenta sair do contrato. O papel dessa feature é provar exatamente isso.

Referências

As referências abaixo foram escolhidas por dois motivos: primeiro, porque capturam a semântica efetiva do ecossistema Docker/containerd/OCI; segundo, porque descrevem diretamente as primitivas de kernel e de integração necessárias para uma implementação realista em Python, Cython e C++. Sempre que possível, privilegiaram-se documentação oficial, especificações e man pages.

[1] Docker Docs. “Configure alternative container runtimes”. Documentação do Docker Engine, descrevendo o uso de containerd e do runc por padrão. URL: <https://docs.docker.com/engine/daemon/alternative-runtimes/> . Acesso em 12 mar. 2026.

[2] containerd. “Containers and shims”. Documentação do projeto containerd sobre o papel do shim como pai direto do processo do contêiner, manutenção de stdio/PTY e reap. URL: <https://github.com/containerd/containerd/blob/main/docs/containers-and-shims.md> . Acesso em 12 mar. 2026.

[3] containerd. “Runtime v2 README”. Observações sobre shim como subreaper e a necessidade de init interno em novos PID namespaces. URL: <https://github.com/containerd/containerd/blob/main/core/runtime/v2/README.md> . Acesso em 12 mar. 2026.

[4] Open Container Initiative. “runtime-spec README”. Visão geral da especificação de runtime da OCI. URL: <https://github.com/opencontainers/runtime-spec/blob/main/README.md> . Acesso em 12 mar. 2026.

[5] Open Container Initiative. “config.md”. Definição de bundle, processo e campos de configuração do contêiner. URL: <https://github.com/opencontainers/runtime-spec/blob/main/config.md> . Acesso em 12 mar. 2026.

[6] Open Container Initiative. “config-linux.md”. Detalhes Linux da configuração e da troca de estado por socket UNIX/SCM_RIGHTS. URL: <https://github.com/opencontainers/runtime-spec/blob/main/config-linux.md> . Acesso em 12 mar. 2026.

- [7] Docker Docs. “Running containers”. Referência do comando docker run, incluindo -i, -t, --init, --pids-limit e opções de recursos. URL: <https://docs.docker.com/engine/containers/run/> . Acesso em 12 mar. 2026.
- [8] Docker Docs. “docker container attach”. Documentação de attach, sig-proxy e observações sobre processos PID 1 no contêiner. URL: <https://docs.docker.com/reference/cli/docker/container/attach/> . Acesso em 12 mar. 2026.
- [9] Docker Docs. “docker container stop”. Semântica de stop signal, timeout e escalonamento para SIGKILL. URL: <https://docs.docker.com/reference/cli/docker/container/stop/> . Acesso em 12 mar. 2026.
- [10] Docker Docs. “Dockerfile reference”. Seção sobre ENTRYPOINT/CMD e implicações do uso de shell form para sinais e PID 1. URL: <https://docs.docker.com/reference/dockerfile/> . Acesso em 12 mar. 2026.
- [11] Docker Docs. “Resource constraints”. Limites de memória, considerações sobre OOM e uso cuidadoso de oom-kill-disable e oom-score-adj. URL: https://docs.docker.com/engine/containers/resource_constraints/ . Acesso em 12 mar. 2026.
- [12] Linux Kernel Documentation. “Control Group v2”. Arquivos memory.high, memory.max, memory.events, memory.oom.group, pids.max, pids.events e cgroup.kill. URL: <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html> . Acesso em 12 mar. 2026.
- [13] Michael Kerrisk. “pid_namespaces(7)”. Semântica do PID 1, adoção de órfãos e sinais em PID namespaces. URL: https://man7.org/linux/man-pages/man7/pid_namespaces.7.html . Acesso em 12 mar. 2026.
- [14] Michael Kerrisk. “PR_SET_CHILD_SUBREAPER(2const)”. Uso de subreaper para reparenting de processos órfãos. URL: https://man7.org/linux/man-pages/man2/PR_SET_CHILD_SUBREAPER.2const.html . Acesso em 12 mar. 2026.
- [15] Michael Kerrisk. “pidfd_open(2)”. File descriptors de processo, espera por término e monitoramento livre de corridas de PID. URL: https://man7.org/linux/man-pages/man2/pidfd_open.2.html . Acesso em 12 mar. 2026.
- [16] Michael Kerrisk. “pidfd_send_signal(2)”. Entrega de sinais via pidfd. URL: https://man7.org/linux/man-pages/man2/pidfd_send_signal.2.html . Acesso em 12 mar. 2026.
- [17] Michael Kerrisk. “signalfd(2)”. Transformação de sinais em eventos lidos via file descriptor. URL: <https://man7.org/linux/man-pages/man2/signalfd.2.html> . Acesso em 12 mar. 2026.
- [18] Michael Kerrisk. “pty(7)”. Semântica de pseudo-terminais, lados master/slave e comportamento terminal. URL: <https://man7.org/linux/man-pages/man7/pty.7.html> . Acesso em 12 mar. 2026.
- [19] Michael Kerrisk. “openpty(3)”. Criação do par PTY e configuração inicial de terminal e janela. URL: <https://man7.org/linux/man-pages/man3/openpty.3.html> . Acesso em 12 mar. 2026.
- [20] krallin/tini. README oficial do Tini, descrevendo o papel do init mínimo em reaping de zombies e forwarding de sinais. URL: <https://github.com/krallin/tini> . Acesso em 12 mar. 2026.
- [21] Cython Documentation. “Wrapping C++”. Declaração de classes C++, headers externos e tradução de exceções. URL: https://cython.readthedocs.io/en/latest/src/userguide/wrapping_CPlusPlus.html . Acesso em 12 mar. 2026.
- [22] Cython Documentation. “Interfacing with External C Code”. Uso de cdef extern from, headers e integração com código nativo. URL: https://cython.readthedocs.io/en/latest/src/userguide/external_C_code.html . Acesso em 12 mar. 2026.
- [23] Cython Documentation. “Source Files and Compilation”. Geração de módulos C/C++, opções de compilação e organização de extensão. URL: https://cython.readthedocs.io/en/latest/src/userguide/source_files_and_compilation.html . Acesso em 12 mar. 2026.